

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №1
дисциплины «Объектно-ориентированное программирование»
Вариант 7

Выполнила:
Еремина Татьяна Евгеньевна
3 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника», очная
форма обучения

(подпись)

Проверил:
Воронкин Р.А., доцент департамента
цифровых, робототехнических систем
и электроники института
перспективной инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: модули и пакеты.

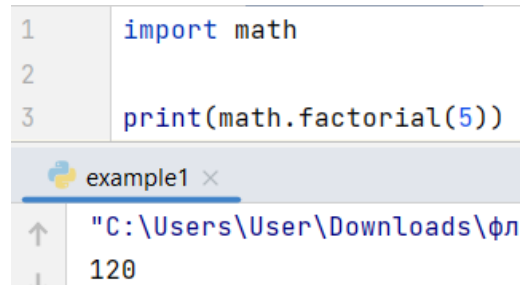
Цель: приобретение навыков по работе с модулями и пакетами языка программирования Python версии 3.x.

Репозиторий: <https://github.com/TL-hash/lab1-OOP>

Практическая часть:

Примеры.

1. Импорт и использование модуля `math`, который содержит математические конструкции.

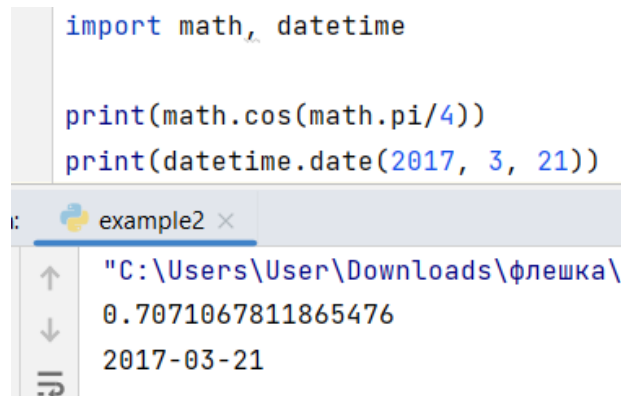


```
1 import math
2
3 print(math.factorial(5))
```

The screenshot shows a code editor with three lines of Python code. Below the code, a terminal window titled 'example1' displays the output of the program: '120'. The file path shown in the terminal is 'C:\Users\User\Downloads\фл'.

Рисунок 1 - Использование модуля `math`

2. За один раз можно импортировать сразу несколько модулей, для этого их нужно перечислить через запятую после слова `import`.



```
import math, datetime

print(math.cos(math.pi/4))
print(datetime.date(2017, 3, 21))
```

The screenshot shows a code editor with four lines of Python code. Below the code, a terminal window titled 'example2' displays the output of the program: '0.7071067811865476' and '2017-03-21'. The file path shown in the terminal is 'C:\Users\User\Downloads\флешка\'.

Рисунок 2 - Использование нескольких модулей

3. Если необходимо задать псевдоним для модуля, можно воспользоваться соответствующим синтаксисом.

```
import math as m

print(m.sin(m.pi/3))
```

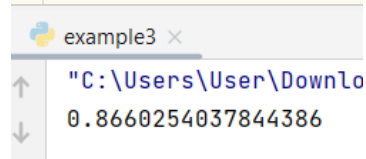


Рисунок 3 - Использование псевдонима для модуля

4. Для того, чтобы не указывать имя модуля при вызове функции из импортированного модуля, необходимо использовать следующую конструкцию.

```
from math import cos

print(cos(3.14))
```

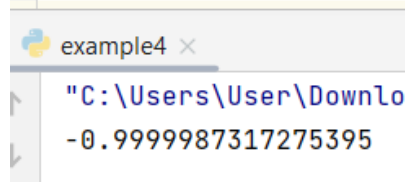


Рисунок 4 - Использование импорта

5. Для импортирования нескольких функций из модуля, можно перечислить их имена через запятую.

```
from math import cos, sin, pi

print(cos(pi/3))
print(sin(pi/3))
```

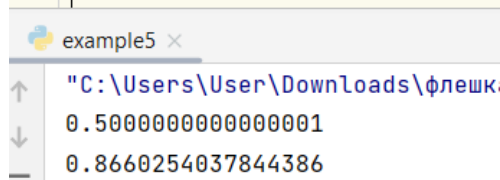


Рисунок 5 - Импорт нескольких функций

6. Импортируемому объекту можно задать псевдоним.

```
from math import factorial as f

print(f(4))
```

example6 x

"C:\Users\User\Downloads\флешка\20
24

Рисунок 6 - Псевдоним импортируемого объекта

7. Если необходимо импортировать все функции, классы и т.п. из модуля, то необходимо воспользоваться *.

```
from math import *

print(cos(pi/2))
print(sin(pi/4))
print(factorial(6))
```

example7 x

"C:\Users\User\Downloa
6.123233995736766e-17
0.7071067811865476
720

Рисунок 7 - Импорт всего из библиотеки

Задания (вариант 7).

Задание 1. Выполнить индивидуальное задание, оформив все функции программы в виде отдельного модуля. Разработанная модель должна быть подключена в основную программу с помощью одного из вариантов import.

Используя замыкания функций, объявите внутреннюю функцию, которая на основе двух параметров вычисляет площадь фигуры. Какой именно фигуры: треугольника или прямоугольника, определяется параметром `type` внешней функции. Если `type` принимает значение 0, то вычисляется площадь треугольника, а иначе – прямоугольника. По умолчанию параметр `type` должен быть равен 0. Вычисленное значение должно возвращаться внутренней функцией. Вызовите внутреннюю функцию замыкания и отобразите на экране результат ее работы.

Решение:

1. Создаётся отдельный модуль `area_module.py`, в котором объявляется внешняя функция `get_area_function(type=0)`.
2. Внешняя функция `get_area_function` принимает один параметр `type` (по умолчанию 0). Внутри неё объявляются две внутренние функции: `triangle_area(base, height)` – вычисляет площадь треугольника по формуле $0.5 * base * height$, `rectangle_area(length, width)` – вычисляет площадь прямоугольника по формуле $length * width$
3. Замыкание: внешняя функция возвращает одну из внутренних функций в зависимости от значения `type`. Если `type == 0`, возвращается `triangle_area`, иначе – `rectangle_area`. Таким образом, возвращаемая функция «замыкается» на контекст, в котором она была создана.
4. Основная программа `main.py` импортирует модуль `area_module` и вызывает внешнюю функцию для получения нужной внутренней функции. Затем внутренняя функция вызывается с двумя аргументами, и результат выводится на экран.
5. Импорт модуля осуществляется одним из стандартных способов:
`import area_module.`

Листинг задания:

Файл `area_module.py` (модуль):

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```
def get_area_function(type=0):
    def triangle_area(base, height):
        return 0.5 * base * height

    def rectangle_area(length, width):
        return length * width

    if type == 0:
        return triangle_area
    else:
        return rectangle_area
```

Файл task1.py (основная программа):

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import area_module

def main():
    triangle_func = area_module.get_area_function()
    print(f"Площадь треугольника: {triangle_func(6, 4)}")

    rectangle_func = area_module.get_area_function(type=1)
    print(f"Площадь прямоугольника: {rectangle_func(5, 3)}")

if __name__ == '__main__':
    main()
```

Результат выполнения:

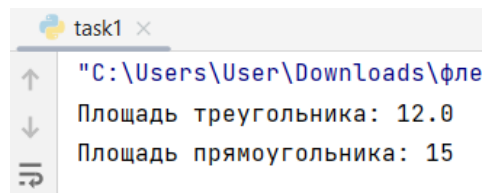


Рисунок 8 - Результат выполнения программы

Задание 2. Выполнить индивидуальное задание, оформив все классы программы в виде отдельного пакета, а каждую команду в виде отдельной функции. Разработанный пакет должен быть подключен в основную программу с помощью одного из вариантов команды `import`. Настроить соответствующим образом переменную `__all__` в файле `__init__.py` пакета.

Составить программу с использованием списков и словарей для решения задачи. Использовать словарь, содержащий следующие ключи: название пункта назначения; номер поезда; время отправления. Написать программу, выполняющую следующие действия: ввод с клавиатуры данных в список, состоящий из словарей заданной структуры; записи должны быть упорядочены по времени отправления поезда; вывод на экран информации о поездах, направляющихся в пункт, название которого введено с клавиатуры; если таких поездов нет, выдать на дисплей соответствующее сообщение.

Решение:

1. Создаётся пакет `train_package`, содержащий четыре модуля: `input_data.py` – функция `input_trains()` для ввода данных с клавиатуры, `sort_data.py` – функция `sort_trains(trains)` для сортировки списка по времени отправления, `search_data.py` – функция `search_by_destination(trains, destination)` для поиска поездов по пункту назначения, `display_data.py` – функция `display_trains(trains)` для вывода информации на экран.

2. В файле `__init__.py` настраивается переменная `__all__`, которая определяет, какие функции будут доступны при импорте `from train_package import *`. Также выполняется относительный импорт этих функций из модулей пакета.

3. Структура данных: список `trains`, каждый элемент которого – словарь с ключами `destination` (пункт назначения), `train_number` (номер поезда), `departure_time` (время отправления).

4. Функция `input_trains` запрашивает количество поездов, затем в цикле собирает данные по каждому поезду и добавляет словарь в список.

5. Функция `sort_trains` использует встроенную функцию `sorted()` с ключом сортировки `lambda train: train['departure_time']`. Так как время в формате чч:мм, строковое сравнение работает корректно.

6. Функция `search_by_destination` проходит по списку и сравнивает пункт назначения без учёта регистра (через `.lower()`). Возвращает список найденных поездов (может быть пустым).

7. Функция `display_trains` выводит данные в виде таблицы. Если список пуст – выводит сообщение.

8. Основная программа `task2.py` импортирует пакет, вызывает функции ввода, сортировки, вывода, поиска и выводит результат поиска или сообщение об отсутствии поездов.

Листинг задания:

Файл `train_package/__init__.py`:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
```

```
__all__ = ['input_trains', 'sort_trains', 'search_by_destination', 'display_trains']
```

```
from .input_data import input_trains
from .sort_data import sort_trains
from .search_data import search_by_destination
from .display_data import display_trains
```

Файл train_package/input_data.py:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

def input_trains():
    trains = []
    n = int(input("Введите количество поездов: "))

    for i in range(n):
        print(f"\nПоезд №{i + 1}:")
        destination = input(" Пункт назначения: ").strip()
        train_number = input(" Номер поезда: ").strip()
        departure_time = input(" Время отправления (ЧЧ:ММ): ").strip()

        train = {
            'destination': destination,
            'train_number': train_number,
            'departure_time': departure_time
        }
        trains.append(train)

    return trains
```

Файл train_package/sort_data.py:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

def sort_trains(trains):
    return sorted(trains, key=lambda train: train['departure_time'])
```

Файл train_package/search_data.py:

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

def search_by_destination(trains, destination):
    found = []
```

```

for train in trains:
    if train['destination'].lower() == destination.lower():
        found.append(train)
return found

```

Файл train_package/display_data.py:

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

def display_trains(trains):
    if not trains:
        print("Нет поездов для отображения.")
        return

    print("\n{:<20}  {:<15}  {:<15}".format("Пункт назначения", "Номер поезда",
"Время отправления"))
    print("-" * 50)
    for train in trains:
        print("{:<20} {:<15} {:<15}".format(
            train['destination'],
            train['train_number'],
            train['departure_time']
        ))

```

Файл task2.py (основная программа):

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import train_package as tp

def main():
    print("Ввод данных о поездах")
    trains = tp.input_trains()

    trains = tp.sort_trains(trains)
    print("\nПоезда, отсортированные по времени отправления")
    tp.display_trains(trains)

    destination = input("\nВведите пункт назначения для поиска: ").strip()
    found = tp.search_by_destination(trains, destination)

    if found:
        print(f"\nПоезда, направляющиеся в пункт '{destination}'")
        tp.display_trains(found)
    else:
        print(f"\nПоездов, направляющихся в пункт '{destination}', не найдено.")

```

```
if __name__ == '__main__':  
    main()
```

Результат выполнения:

```
Ввод данных о поездах  
Введите количество поездов: 2  
  
Поезд №1:  
    Пункт назначения: Ставрополь  
    Номер поезда: 123  
    Время отправления (ЧЧ:ММ): 14:20  
  
Поезд №2:  
    Пункт назначения: Москва  
    Номер поезда: 454  
    Время отправления (ЧЧ:ММ): 17:30  
  
Поезда, отсортированные по времени отправления  
  
Пункт назначения      Номер поезда      Время отправления  
-----  
Ставрополь            123              14:20  
Москва                454              17:30  
  
Введите пункт назначения для поиска: Ставрополь  
  
Поезда, направляющиеся в пункт 'Ставрополь'  
  
Пункт назначения      Номер поезда      Время отправления  
-----  
Ставрополь            123              14:20
```

Рисунок 9 - Результат выполнения программы

Вывод: в ходе лабораторной работы были изучены и практически освоены основные способы подключения модулей в Python 3.x: импорт модуля целиком, импорт нескольких модулей, использование псевдонимов, импорт отдельных функций и импорт всех имён через *.

Были приобретены навыки создания собственных модулей и пакетов, настройки переменной `__all__` в файле `__init__.py`, а также организации

относительного импорта между модулями внутри пакета. На практике закреплены приёмы работы с замыканиями, списками и словарями, сортировкой данных и поиском по заданному критерию.

Контрольные вопросы:

1. Что является модулем языка Python?

Модулем в Python является файл с расширением `.py`, содержащий код на языке Python: функции, классы, переменные и исполняемые инструкции. Модуль можно импортировать в другую программу с помощью команды `import`, чтобы использовать его содержимое повторно. Имя модуля совпадает с именем файла без расширения. Например, файл `math.py` – это модуль `math`. Модули позволяют структурировать код, избегать дублирования и повторно использовать готовые решения.

2. Какие существуют способы подключения модулей в языке Python?

В Python существуют следующие основные способы подключения модулей:

- 1) импорт модуля целиком: `import math`;
- 2) импорт нескольких модулей одновременно: `import math, datetime`;
- 3) импорт модуля с псевдонимом: `import math as m`;
- 4) импорт отдельных объектов из модуля: `from math import cos`;
- 5) импорт нескольких объектов из модуля: `from math import cos, sin, pi`;
- 6) импорт объекта с псевдонимом: `from math import factorial as f`;
- 7) импорт всех имён из модуля: `from math import *`.

Каждый способ применяется в зависимости от задачи: если нужно использовать много функций из модуля – удобнее импортировать модуль целиком; если нужна только одна функция – можно импортировать её напрямую.

3. Что является пакетом языка Python?

Пакетом в Python называется папка (каталог), содержащая файл `__init__.py` и один или несколько модулей (файлов `.py`). Пакет позволяет объединять модули в единую иерархическую структуру и обращаться к ним через точечную нотацию, например: `import train_package.input_data`. Пакеты

используются для организации крупных проектов, где код логически разделён на модули по назначению.

4. Каково назначение файла `__init__.py`?

Файл `__init__.py` находится внутри папки пакета и выполняет несколько функций:

- 1) сообщает интерпретатору Python, что данная папка является пакетом;
- 2) выполняется автоматически при импорте пакета;
- 3) может содержать код инициализации пакета (например, настройку переменных, импорт подмодулей);
- 4) в нём можно задать переменную `__all__`, определяющую, какие имена будут импортированы при использовании `from package import *`.

Файл может быть пустым, но его наличие обязательно (в Python 3.3+ формально не обязательно, но рекомендуется для совместимости и явности).

5. Каково назначение переменной `__all__` файла `__init__.py`?

Переменная `__all__` в файле `__init__.py` представляет собой список имён (строк), которые будут импортированы при выполнении команды `from package import *`. Она определяет публичный интерфейс пакета и позволяет ограничить доступ к внутренним модулям и функциям. Например: `__all__ = ['input_trains', 'sort_trains', 'search_by_destination', 'display_trains']`. Если переменная `__all__` не задана, то при импорте `*` будут импортированы все имена, не начинающиеся с подчёркивания. Настройка `__all__` помогает явно указать, что именно предназначено для внешнего использования, и скрыть детали реализации.